JOIN PIPELINE OPTIMIZATION

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

Τζαφέρης Στέφανος 1115202200183

Γαλιατσάτου Ελευθερία 1115202200025

Όλα τα πειράματα πραγματοποιήθηκαν στα **Linux** μηχανήματα της σχολής, στα οποία προσπαθούσαμε να τρέχουμε τα πειράματα ώρες που δεν υπήρχε μεγάλος φόρτος για την καλύτερη δυνατή απόδοση. Όλοι οι χρόνοι συγκεντρώθηκαν από τους μέσους όρους των χρόνων 4 εκτελέσεων για κάθε πείραμα, προκειμένου να λάβουμε πιο σταθερά αποτελέσματα.

Ενδεικτικά ένα μηχάνημα έχει :

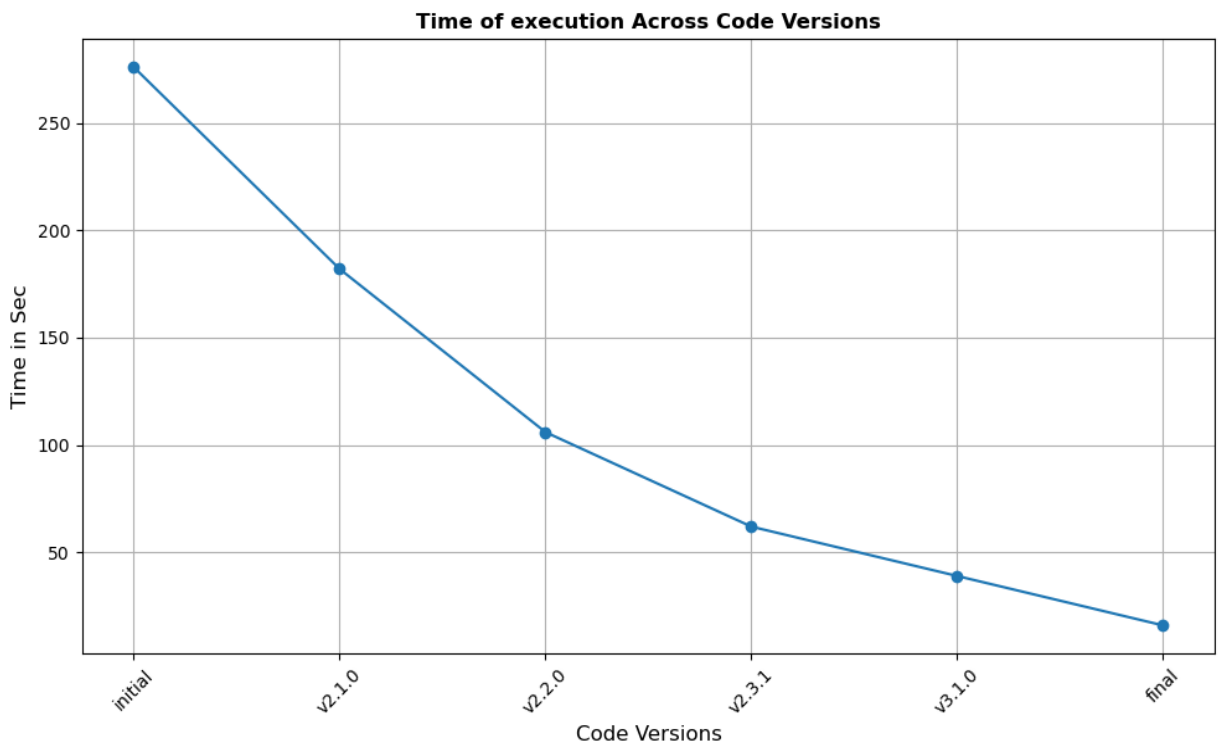
Αριθμός πυρήνων : 4

Επεξεργαστής : Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

OS : NAME="Ubuntu" VERSION="20.04.6 LTS (Focal Fossa)"

Έκδοση μεταγλωττιστή : 9.4.0

Παρακάτω παρουσιάζονται συνοπτικά οι χρόνοι εκτέλεσης καθ' όλη την διάρκεια της υλοποίησης των παραδοτέων 2 και 3 με έμφαση σε αυτούς του τρίτου παραδοτέου 3 της εργασίας.



Βασική λύση

M.O 276637 ms

2.1 Late materialization με παραγωγή ColumnarTable στο top level join

M.O 182186 ms

-34% από τον χρόνο της base

2.2 Column-store δομή

M.O 106616 ms

- 61% από τον χρόνο της base

2.3 Unchained Hash Table

M.O 62.034 ms

-77,5% από τον χρόνο της base

Στο δεύτερο παραδοτέο, υπήρξε μεγάλη βελτίωση γενικότερα κυρίως μέσω της υλοποίησης του unchained hash table σε συνδυασμό με την column-store δομή με την οποία η προσπέλαση στοιχείων ενός πεδίου πραγματοποιείται μόνο μέσω του row index του. Επιπλέον, η προσθήκη της hash function crc32 της βιβλιοθήκης <nmmintrin.h> ήταν επίσης παράγοντας που μείωσε σημαντικά τον χρόνο. Όσον αφορά τα prefix bits, πειραματιστήκαμε με διάφορες τιμές, ωστόσο τα 16 bits (2^16 directory entries) είχαν την καλύτερη απόδοση. Τέλος, ορίσαμε το look up table array ως static προκειμένου να υπολογίζεται μία μόνο φορά, γεγονός που είχε θετική επίδραση στον χρόνο.

Γενικότερα, χρησιμοποιήθηκαν όπου ήταν δυνατό, const, inline functions και moves για την αποφυγή αντιγραφής κυρίως μεγάλων αντικειμένων.

3.1 Βελτιστοποίηση indexing

Στο πρώτο αυτό ζητούμενο του τρίτου παραδοτέου, η Column-store δομή κρατάει δύο επιπλέον πληροφορίες για μία στήλη με σκοπό την αποφυγή αντιγραφής dense στηλών, δηλαδή με πεδία int32 που δεν εμπεριέχουν null τιμές:

- Boolean μεταβλητή copied, που παίρνει την τιμή true εάν η στήλη έχει null τιμές και επομένως έχει αντιγραφεί, ή την τιμή false στην περίπτωση που είναι dense.
- Const Column* colRef, που είναι ένας δείκτης σε στήλη τύπου Column. Ο δείκτης αυτός είναι έγκυρος και αξιοποιείται μόνο για τις dense στήλες με copied = false.

Πλέον, με την υλοποίηση αυτή η προσπέλαση των σελίδων που ανήκουν σε dense στήλες γίνεται μέσω της αρχικής στήλης. Για τον σκοπό αυτό, αξιοποιείται το υπάρχον vector pageRowOffset<size_t> που κρατάει τον συνολικό αριθμό γραμμών έως την αρχή κάθε σελίδας. Για παράδειγμα pageRowOffset[0] = 0, pageRowOffset[1] = αριθμός γραμμών πρώτης σελίδας κ.ο.κ. Έτσι, όταν χρειάζεται να προσπελάσουμε κάποιο στοιχείο γνωρίζοντας τη γραμμή του, μπορούμε με την χρήση της upper_bound() συνάρτησης εύκολα να εντοπίσουμε το κατάλληλο offset γραμμών από το vector και να ανακτήσουμε το στοιχείο σε max O(logn) (η συνάρτηση αυτή χρησιμοποιεί binary search) .

M.O: 39256 ms

-85% από τον χρόνο της base

3.2 Build phase παραλληλοποίηση – χωρίς slab allocator

Για την παραλληλοποίηση του build, έχει δημιουργηθεί μια δομή ThreadLocalData (στη συνέχεια αντικαθίσταται λόγω του slab allocator με τους collectors) την οποία κρατάει κάθε thread και στην οποία αποθηκεύονται όλα τα partitions προκειμένου κάθε νήμα να μπορεί να γράψει στα «δικά του» partitions για αποφυγή συγχρονισμού. Μέσα στο ίδιο το hash table, αποθηκεύονται σε ένα vector local_data τόσα ThreadLocalData όσα και τα νήματα. Στη δομή αυτή εισάγονται όλα τα tuples πριν τη δημιουργία των directory entries.

M.O: 39257 ms

- 85% από τον χρόνο της base

3.3.1 Probe phase παραλληλοποίηση

Η παραλληλοποίηση του probe phase του hash join αλγορίθμου, είναι εφικτή μέσω δύο βασικών δομών που επιτρέπουν στα threads να συλλέγουν τα αποτελέσματα τους σε ένα τοπικό table στο οποίο μπορεί να γράφει ένα μοναδικό thread καθ' όλη τη διάρκεια του probing. Κάθε ένα από αυτά τα threads έχει στη διάθεση του επίσης «τοπικούς» inserters, δηλαδή δομές που πραγματοποιούν την εισαγωγή στοιχείων σε μία συγκεκριμένη στήλη. Στο τέλος,

της συγκέντρωσης των στοιχείων στον τοπικό πίνακα κάθε thread, συγχωνεύονται από ένα μοναδικό thread τα τοπικά αυτά αποτελέσματα σε έναν ενιαίο πίνακα. Οι δύο δομές που αξιοποιήθηκαν για τον σκοπό αυτό και αναφέρθηκαν παραπάνω είναι οι εξής:

- vector<vector<ColumnT>> threadResults: αναπαριστά n πίνακες όσους ο αριθμός των νημάτων. Κάθε νήμα έχει τις δικές του στήλες στις οποίες γράφει.
- vector<vector<ColumnTInserter>> threadInserter: κρατάει τους inserters για όλες τις στήλες του κάθε νήματος.

M.O: 19496 ms

-92% από τον χρόνο της base

3.3.2 Work stealing

Το work stealing εφαρμόστηκε και κατά τη διάρκεια του probe αλλά και του build phase. Συγκεκριμένα, το parallel for loop της OMP αντικαταστάθηκε με ένα απλό παράλληλο κομμάτι κώδικα στο οποίο κατανέμονται οι σελίδες εντός ενός while loop. Στο loop αυτό, αναθέτονται κάθε φορά 4 σελίδες σε κάθε νήμα μέσω ενός atomic counter. Όταν κάποιο νήμα ολοκληρώσει το δικό του κομμάτι εργασίας, μπορεί να αναλάβει κάποιο επόμενο μερίδιο μπαίνοντας εκ νέου στο while loop και αυξάνοντας τον counter έως ότου εξαντληθούν όλες οι σελίδες. Με τον τρόπο αυτό, εάν κάποιο νήμα καθυστερεί ή οι σελίδες που έχει αναλάβει έχουν μεγαλύτερο φόρτο, τα υπόλοιπα νήματα δεν εξαρτώνται από αυτό και μπορούν να εκτελέσουν επιπλέον έργο.

M.O 17120 ms

-93,8% από τον χρόνο της base

Slab allocator (υλοποιήθηκε τελευταίος):

Στο τέλος αφού είχαν υλοποιηθεί όλα τα ζητούμενα της εργασίας, προσπαθήσαμε να φτιάξουμε την λογική του slab allocator με 3 level όπου το πρώτο στην πραγματικότητα δεσμεύει μνήμη και τα άλλα ζητάν από το υψηλότερο επίπεδο όπως αναφέρει στην εκφώνηση. Ο slab allocator χρησιμοποιήθηκε στην Insert (η consume function) καθώς και στην merge partitions στην συλλογή των tuples όταν ενώνονται τα tuples των δομών του slab allocator. Πέρα από τα σωστά αποτελέσματα, η υλοποίηση του slab allocator δεν μας δίνει κάποια σημαντική μείωση σε χρόνο με τον τρόπο που την υλοποιήσαμε τουλάχιστον.

M.O 17056 ms

-93,8 από τον χρόνο της base

* Στα πλαίσια βελτίωσης χρόνου δοκιμάσαμε και διαφορετικές hash function(Fibonacci hash, murmurHash, xxHash) άλλα δεν υπήρξε κάποια βελτίωση. Επίσης, προσπαθήσαμε μετά την κλήση της τελικής hash function crc32 να προσθέσουμε κάποιες επιπλέον πράξεις bit για ακόμη καλύτερη κατανομή, αλλά επίσης δεν παρατηρήσαμε διαφορά.

Σύγκριση partitions με αριθμό νημάτων

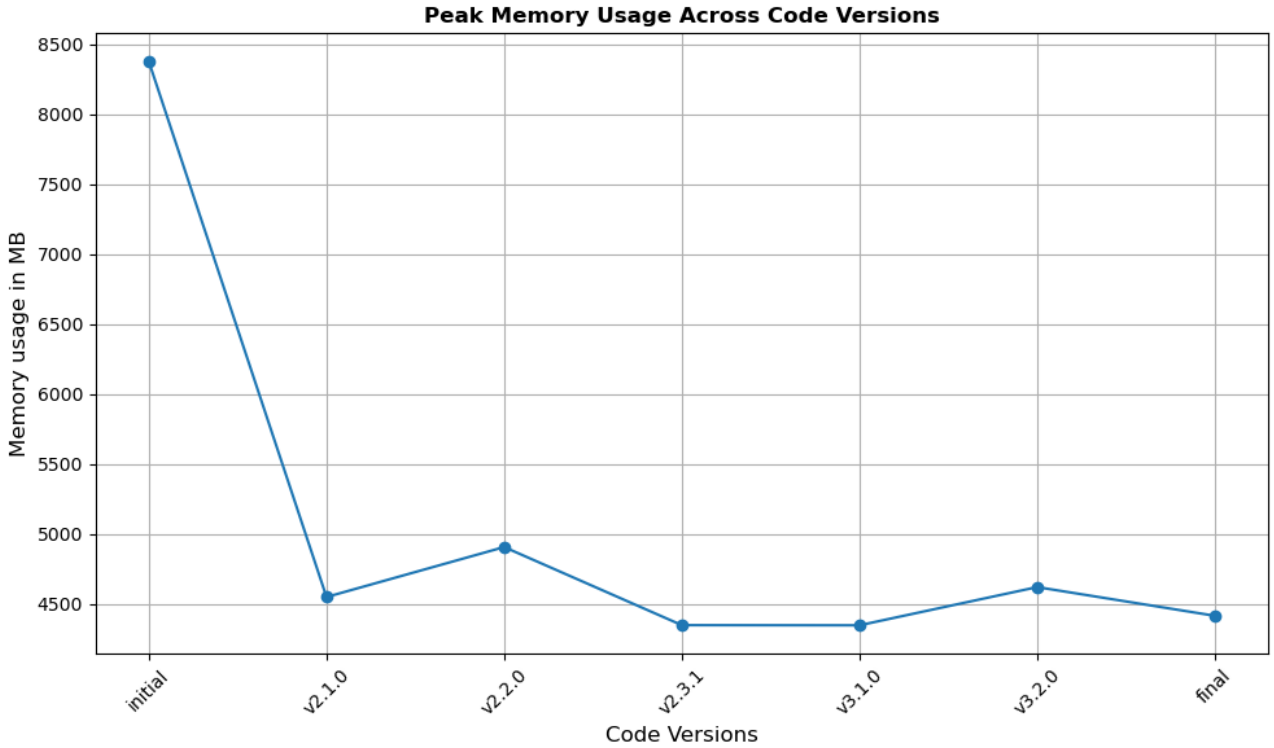
Τα παρακάτω πειράματα για την εξέταση διαφορετικού αριθμού partitions σε σχέση με αριθμό threads, πραγματοποιήθηκαν με σταθερό αριθμό prefix bits ορισμένο σε 16. (Τα αποτελέσματα διαφέρουν ελάχιστα από τα παραπάνω γιατί τα πειράματα εκτελέστηκαν σε διαφορετική στιγμή)

Πλήθος partition bits	**Πλήθος νημάτων**			
	1	2	4	8
2 bits 4 partitions	39438 ms	26423 ms	18041 ms	18178 ms
3 bits 8 partitions	40244 ms	26261 ms	17917 ms	18182 ms
4 bits 16 partitions	40900 ms	25610 ms	17990 ms	18156 ms
5 bits 32 partitions	40839 ms	25758 ms	18111 ms	18123 ms

Όπως προκύπτει από τον πίνακα, η εκτέλεση του αλγορίθμου με ένα μόνο νήμα προφανώς δεν επιδράει στο συνολικό χρόνο εκτέλεσης, εφόσον δεν υπάρχει πράγματι παραλληλία. Μάλιστα, το ένα νήμα απαιτεί ελάχιστα περισσότερο χρόνο από τη σειριακή εκτέλεση, λόγω της αρχικοποίησης των πόρων για την OpenMP. Στη συνέχεια, για 2 και 4 νήματα παρατηρείται η μεγαλύτερη βελτίωση, ενώ από 4 σε 8 νήματα ο χρόνος κυμαίνεται στο ίδιο εύρος τιμών. Αυτό οφείλεται στο γεγονός ότι τα linux μηχανήματα στα οποία έτρεξαν τα πειράματα, διαθέτουν 4 πυρήνες οπότε οτιδήποτε πάνω από 4 νήματα δεν έχει ιδιαίτερη επιρροή στην

απόδοση και την επιβαρύνει ελάχιστα με το overhead για την εναλλαγή των νημάτων. Όσον αφορά τα partitions, τα 8 partitions σε συνδυασμό με τα 4 νήματα παρουσιάζουν την βέλτιστη απόδοση (χωρίς αυτή να διαφοροποιείται ιδιαίτερα για τις διάφορες παραμέτρους). Τελικά, διατηρήθηκε ο συνδυασμός 8 partitions και 8 νήματα, όπως ζητείται άλλωστε και στην εκφώνηση.

Κατανάλωση μνήμης



**Το version v3.2.0 είναι το μόνο που δεν υπάρχει στο github ως tag, και η μόνη διαφορά από την τελική λύση είναι ότι δεν είχε υλοποιηθεί η λογική του slab allocator ακόμα*

Όπως είναι φανερό στο πόση μνήμη καταναλώνει στην χειρότερη στιγμή το πρόγραμμα μας (peak memory usage) το late materialization μειώνει την χρήση μνήμης σχεδόν στο μισό. Έπειτα από αυτό δεν υπάρχουν σημαντικές διακυμάνσεις σε άλλες version του κώδικα.

Η παρακάτω συνάρτηση χρησιμοποιήθηκε στην read_sql.cpp για να μπορούμε να δούμε το max memory usage κατά την εκτέλεση όλων των ερωτημάτων.

```
#include <string>
void print_memory_usage() {
    struct rusage usage;
    if (getrusage(RUSAGE_SELF, &usage) == 0) {
        // Peak resident set size (in kilobytes)
        long peak_kb = usage.ru_maxrss;
        std::cout << "Peak memory usage: " << peak_kb << " KB (≈ "
                  << peak_kb / 1024.0 << " MB)" << std::endl;
    } else {
        perror("getrusage");
    }
}
```